Code Explanation

Delta Live Tables Pipeline Code Explanation

This code is a Delta Live Tables (DLT) pipeline written in Python, which is used to process and transform data in a Databricks environment. The pipeline is designed to handle customer and order data, applying various transformations and creating aggregated tables.

Overview of the Code
The code defines a series of DLT tables that read data from CSV files, clean and transform the data, and then apply Slowly Changing Dimension (SCD) Type 2 updates to create a current version of the data. The pipeline also creates an aggregated table to show customer spending.

Step 1: Importing Necessary Libraries and Defining Source Tables
This step imports the required libraries and defines the source tables for customer and order data.

```
import dlt
from pyspark.sql.functions import col, lit, current_timestamp, when,
 datediff, to_date, sum as sum_

# Define the source tables
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from source"
)
def customer_bronze():
    return spark.read.format("csv")
        .option("header", "true")
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/custome
rdata")

@dlt.table(
    name="order_bronze",
    comment="Raw order data from source"
)
def order_bronze():
    return spark.read.format("csv")
        .option("header", "true")
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderda
ta")
```

Step 2: Cleaning and Transforming Customer and Order Data
This step cleans and transforms the customer and order data by removing duplicates and null values.

```
# Clean and transform customer data
@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data"
)
def customer_silver():
    return dlt.read("customer_bronze")
        .dropDuplicates()
        .na.drop()
```

```
# Clean and transform order data
@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return dlt.read("order_bronze")
        .dropDuplicates()
        .na.drop()
        .withColumn("PricePerUnit", col("PricePerUnit").cast("double
"))
        .withColumn("Qty", col("Qty").cast("integer"))
        .withColumn("Date", col("Date").cast("date"))
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
```

Step 3: Creating the Initial ordersummary Table with SCD Type 2 Columns
This step creates the initial `ordersummary` table with SCD Type 2 columns.

```
# Create SCD Type 2 ordersummary table
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table with customer and order data",
    table_properties={
        "delta.enableChangeDataFeed": "true",
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true"
    }
)
@dlt.expect_or_drop("valid_customer_id", "CustId IS NOT NULL")
@dlt.expect_or_drop("valid_order_id", "OrderId IS NOT NULL")
def ordersummary():
    # Get current customer and order data
    customer_df = dlt.read("customer_silver")
    order_df = dlt.read("order_silver")

    # Join customer and order data
    joined_df = customer_df.join(order_df, "CustId")

    # Add SCD Type 2 columns
    result_df = joined_df.select(
        "CustId", "Name", "EmailId", "Region", "OrderId", "ItemName"
,
        "PricePerUnit", "Qty", "Date", "TotalAmount"
    ).withColumn("IsActive", lit(True))
     .withColumn("StartDate", current_timestamp())
     .withColumn("EndDate", lit(None).cast("timestamp"))

    return result_df
```

Step 4: Handling SCD Type 2 Updates
This step creates a temporary table `ordersummary_updates` to handle SCD Type 2 updates by identifying changed customer records and marking them as inactive.

```
# Create a function to handle SCD Type 2 updates
@dlt.table(
    name="ordersummary_updates",
    comment="Updates to the ordersummary table based on changes in c
ustomer data",
    temporary=True
)
def ordersummary_updates():
    # Get the current active records
    current_records = dlt.read("ordersummary").filter(col("IsActive"
) == True)

    # Get the latest customer data
    new_customer_data = dlt.read("customer_silver")

    # Find records where customer data has changed
    changed_records = current_records.join(
        new_customer_data,
        (current_records.CustId == new_customer_data.CustId) &
        (
            (current_records.Name != new_customer_data.Name) |
            (current_records.EmailId != new_customer_data.EmailId) |
            (current_records.Region != new_customer_data.Region)
        ),
        "inner"
    ).select(current_records["*"])

    # Mark changed records as inactive
    return changed_records.withColumn("IsActive", lit(False))
                          .withColumn("EndDate", current_timestamp())
```

Step 5: Applying SCD Type 2 Updates to the ordersummary Table

This step applies the SCD Type 2 updates to the `ordersummary` table by creating a new table `ordersummary_current`.

```
# Apply the updates to the ordersummary table
@dlt.table(
    name="ordersummary_current",
    comment="Current version of the ordersummary table with SCD Type
 2 updates applied"
)
def ordersummary_current():
    # Get the current ordersummary data
    ordersummary_df = dlt.read("ordersummary")

    # Get the updates
    updates_df = dlt.read("ordersummary_updates")

    # Apply the updates - mark records as inactive
    updated_records = ordersummary_df.join(
        updates_df.select("CustId", "OrderId"),
        ["CustId", "OrderId"],
        "left_anti"
    )

    # Add the inactive records
    result_df = updated_records.union(updates_df)
```

```
    # Add new records for the changed customers
    customer_df = dlt.read("customer_silver")
    order_df = dlt.read("order_silver")

    # Get the customer IDs that have changed
    changed_customer_ids = updates_df.select("CustId").distinct()

    # Create new records for these customers
    new_records = customer_df.join(
        changed_customer_ids,
        "CustId",
        "inner"
    ).join(
        order_df,
        "CustId",
        "inner"
    ).select(
        customer_df.CustId, customer_df.Name, customer_df.EmailId, c
ustomer_df.Region,
        order_df.OrderId, order_df.ItemName, order_df.PricePerUnit,
order_df.Qty,
        order_df.Date, order_df.TotalAmount
    ).withColumn("IsActive", lit(True))
     .withColumn("StartDate", current_timestamp())
     .withColumn("EndDate", lit(None).cast("timestamp"))

    # Combine all records
    return result_df.union(new_records)
```

Step 6: Creating the customeraggregatespend Table
This step creates the `customeraggregatespend` table by aggregating the `TotalAmount` by `Name` and `Date` from the `ordersummary_current` table.

```
# Create the customeraggregatespend table
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date"
)
def customeraggregatespend():
    # Get active records from ordersummary
    active_records = dlt.read("ordersummary_current").filter(col("Is
Active") == True)

    # Aggregate TotalAmount by Name and Date
    return active_records.groupBy("Name", "Date")
                         .agg(sum_("TotalAmount").alias("TotalAmount")
)
```